Web Application Development

Lecture 6: React Forms

SWE 381 – Web Application Development

Dr. Ohoud Alharbi Dr. Achraf Gazdar

Department of Software Engineering
College of Computer and Information Sciences
King Saud University

Term 2 – 2025/2026

Agenda

1. Controlled vs Uncontrolled Inputs
2. All Input Types
3. One State Object + Generic Handler
4. onSubmit + e.preventDefault()
5. Per-Field Validation
6. Reset After Submit
7. Complete Form Example
8. Recap

Outline

Learning Objectives

By the end of this lecture, you will be able to:

- Distinguish between **controlled** and **uncontrolled** input components
- Implement controlled versions of **all major input types**
- Use **one state object** and a **generic handler** to manage multi-field forms
- Handle form submission with `onSubmit` and prevent page reloads
- Build **per-field validation** with error messages
- **Reset** all form fields to initial state after a successful submission

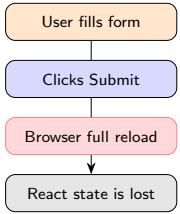
How HTML Forms Work – The Default Behaviour

W3Schools – React Forms

Just like in HTML, React uses forms to allow users to interact with the web page. A standard HTML form will **submit and refresh the page**. In React, we want to prevent this default behaviour and let React **control the form**.

Plain HTML form (traditional):

```
{/* Plain HTML -- works but refreshes! */}
function MyForm() {
  return (
    <form>
      <label>
        Enter your name:
        <input type="text" />
      </label>
    </form>
  );
}
```



Problem: The form submits and the page refreshes – React loses all state.

React solution

Use `onSubmit` + `e.preventDefault()` to intercept the form and handle the data in JavaScript.

Controlled vs Uncontrolled – Side by Side

Uncontrolled input:

```jsx
{/* No value prop -- DOM owns the state */}
function UncontrolledForm() {
  function handleSubmit(e) {
    e.preventDefault();
    // Read value only at submit time
    alert(e.target.myInput.value);
  }
  return (
    <form onSubmit={handleSubmit}>
      <input name="myInput" />
      <button type="submit">Send</button>
    </form>
  );
}
```

DOM tracks the value – React has **no idea** what's in the field until submit.

Controlled input (recommended):

```jsx
import { useState } from 'react';

function ControlledForm() {
  const [name, setName] = useState("");

  function handleSubmit(e) {
    e.preventDefault();
    // React state always has the value
    alert("Hello " + name);
  }
  return (
    <form onSubmit={handleSubmit}>
      <input
        value={name}
        onChange={e => setName(e.target.value)}
      />
      <button type="submit">Send</button>
    </form>
  );
}
```

React state is the **single source of truth** at every keystroke.

Controlled vs Uncontrolled – Comparison

Feature	Uncontrolled	Controlled
Who owns the value	The DOM	React state
Value access timing	Only at submit	Anytime
Real-time validation	Difficult	Easy
Instant UI feedback	Not possible	Possible
Reset form	Manual DOM reset	Set state to initial
Conditional disable	Difficult	Easy
Code amount	Less boilerplate	More (but cleaner)
W3Schools recommendation	For simple cases	**Recommended**

W3Schools

In React, mutable state is typically kept in the state property of components, and only updated with the setter function. This makes the React state the **"single source of truth"**.

When to use each

Use **controlled** components for: validation, conditional disabling, real-time feedback, resetting. Use **uncontrolled** only for simple file upload inputs.

First Controlled Input – W3Schools Example

W3Schools – React Forms

We can use the useState Hook to keep track of each input value and provide a **single source of truth** for the entire application.

```jsx
import { useState } from 'react';

function MyForm() {
  const [name, setName] = useState("");

  function handleChange(e) { setName(e.target.value); }

  return (
    <form>
      <label>
        Enter your name:
        <input type="text" value={name} onChange={handleChange}
      />
      </label>
      <p>Current value: {name}</p>
    </form>
  );
}
```

As user types "Ahmed"

Enter your name:

Ahmed

Current value: **Ahmed**

The 3-part contract:

1. `value={name}` – React controls display
2. `onChange={handleChange}` – fires on every key
3. `setName(e.target.value)` – state updated

value without onChange

If you set `value` without `onChange`, the input becomes read-only. React will warn you.

Outline

Controlled Text and Number Inputs

Text, email, password, number:

```jsx
import { useState } from 'react';

function InputTypes() {
  const [name,     setName]     = useState("");
  const [email,    setEmail]    = useState("");
  const [password, setPassword] = useState("");
  const [age,      setAge]      = useState(18);

  return (
    <form>
      <label>Name:
        <input type="text" value={name}
          onChange={e => setName(e.target.value)} />
      </label>
      <label>Email:
        <input type="email" value={email}
          onChange={e => setEmail(e.target.value)} />
      </label>
      <label>Password:
        <input type="password" value={password}
          onChange={e => setPassword(e.target.value)} />
      </label>
      <label>Age:
        <input type="number" value={age} min="1" max="120"
          onChange={e => setAge(Number(e.target.value))} />
      </label>
    </form>
  );
}
```

Browser Rendering

Name:

Email:

Password:

Age:

18 ↑↓

Number inputs

`e.target.value` is always a **string** – use `Number(...)` or `parseInt(...)` to convert numeric fields before storing in state.

Controlled Textarea – W3Schools

W3Schools – React Textarea

In HTML, <textarea> value is set via children. In React, you use the value attribute instead – just like a regular input. This makes it consistent with all other form controls.

```jsx
import { useState } from 'react';

function MyForm() {
  const [textarea, setTextarea] =
    useState("Initial content here...");

  const handleChange = (e) => { setTextarea(e.target.value); };

  return (
    <form>
      <label>Comments:</label>
      {/* React: use value= not children */}
      <textarea value={textarea} onChange={handleChange}
        rows={4} cols={40} />
      <p>Characters: {textarea.length}</p>
    </form>
  );
}
```

Browser Rendering

Comments:

Initial content here...

Characters: **23**

HTML	React (JSX)
<textarea>text</textarea>	value="text"
Self-closing not allowed	<textarea /> ok

Controlled Select Dropdown – W3Schools

W3Schools – React Select

In React, the selected `<option>` is controlled via the `value` attribute on the `<select>` element itself (not via the `selected` attribute on each option).

```jsx
import { useState } from 'react';

function MyForm() {
  const [dept, setDept] = useState("SWE");

  const handleSubmit = (e) => {
    e.preventDefault();
    alert("Selected dept: " + dept);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Pick your department:
        {/* value= on <select>, not <option> */}
        <select value={dept}
          onChange={e => setDept(e.target.value)}>
          <option value="SWE">Software Engineering</option>
          <option value="CS">Computer Science</option>
          <option value="IS">Information Systems</option>
        </select>
      </label>
      <button type="submit">Submit</button>
      <p>Selected: {dept}</p>
    </form>
  );
}
```

Browser Rendering

Pick your department:

Software Engineering ∨

Submit

Selected: **SWE**

HTML	React
`<option selected>`	`<select value={x}>`

Initial value

The `useState("SWE")` sets the **pre-selected option**. The dropdown opens with SWE already chosen.

Controlled Checkbox – W3Schools

W3Schools – React Checkbox

Checkboxes use the `checked` attribute instead of `value` to control their state. In the `handleChange` function, check `e.target.type === 'checkbox'` to read `e.target.checked` instead of `e.target.value`.

```jsx
import { useState } from 'react';

function AgreementForm() {
  const [agreed, setAgreed] = useState(false);
  const [subscribe, setSubscribe] = useState(true);

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!agreed) { alert("Please accept the terms!"); return; }
    alert("Submitted! Subscribe: " + subscribe);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        <input type="checkbox" checked={agreed}
          onChange={e => setAgreed(e.target.checked)} />
        I agree to the Terms and Conditions
      </label>
      <br />
      <label>
        <input type="checkbox" checked={subscribe}
          onChange={e => setSubscribe(e.target.checked)} />
        Subscribe to newsletter
      </label>
      <br />
      <button type="submit">Submit</button>
    </form>
  );
}
```

Browser Rendering

☐ I agree to the Terms and Conditions

☒ Subscribe to newsletter

Submit

Attribute	Use for
value	text, email, select
checked	checkbox, radio

e.target.checked

For checkboxes, always read `e.target.checked` (boolean), not `e.target.value`.

Controlled Radio Buttons – W3Schools

W3Schools – React Radio

Radio buttons use checked set to a comparison expression, so only the one matching the current state is checked. All radio buttons in a group share the same name and the same onChange handler.

```jsx
import { useState } from 'react';

function MyForm() {
  const [selectedFruit, setSelectedFruit] = useState('banana');

  const handleChange = (e) => { setSelectedFruit(e.target.value); };

  const handleSubmit = (e) => {
    e.preventDefault();
    alert("Favorite fruit: " + selectedFruit);
  };

  return (
    <form onSubmit={handleSubmit}>
      <p>Select your favorite fruit:</p>
      <label>
        <input type="radio" name="fruit" value="apple"
          checked={selectedFruit === 'apple'}
          onChange={handleChange} /> Apple
      </label><br />
      <label>
        <input type="radio" name="fruit" value="banana"
          checked={selectedFruit === 'banana'}
          onChange={handleChange} /> Banana
      </label><br />
      <label>
        <input type="radio" name="fruit" value="cherry"
          checked={selectedFruit === 'cherry'}
          onChange={handleChange} /> Cherry
      </label><br />
      <button type="submit">Submit</button>
    </form>
  );
}
```

selectedFruit = banana

Select your favorite fruit:

- ○ Apple
- ● Banana
- ○ Cherry

Submit

checked = comparison

The pattern checked={selectedFruit === 'banana'} evaluates to true for the matching option and false for all others – only one can be selected.

Input Types Summary

Input type	JSX element	State	Control	Event value
Text	`<input type="text">`	`string`	`value`	`e.target.value`
Email	`<input type="email">`	`string`	`value`	`e.target.value`
Password	`<input type="password">`	`string`	`value`	`e.target.value`
Number	`<input type="number">`	`number`	`value`	`Number(e.target.value)`
Textarea	`<textarea>`	`string`	`value`	`e.target.value`
Select	`<select>`	`string`	`value`	`e.target.value`
Checkbox	`<input type="checkbox">`	`boolean`	`checked`	`e.target.checked`
Radio	`<input type="radio">`	`string`	`checked`	`e.target.value`

Three special rules

(1) **Checkbox/radio**: use `checked={}` not `value={}` for the control attribute.
(2) **Checkbox**: read `e.target.checked` not `e.target.value`.
(3) **Number**: convert with `Number()` – `e.target.value` is always a string.

Outline

Multiple Inputs – W3Schools Pattern

W3Schools – React Multiple Inputs

Using a **single** `useState` object for all fields. The `handleChange` function uses the `name` attribute to update only the changed field.

```jsx
import { useState } from 'react';
function MyForm() {
  const [inputs, setInputs] = useState({});
  const handleChange = (e) => {
    const name  = e.target.name;
    const value = e.target.value;
    setInputs(values => ({ ...values, [name]: value }));
  };
  return (
    <form>
      <label>First name:
        <input type="text" name="firstname"
          value={inputs.firstname || ""}
          onChange={handleChange} />
      </label>
      <label>Last name:
        <input type="text" name="lastname"
          value={inputs.lastname || ""}
          onChange={handleChange} />
      </label>
      <p>Values: {inputs.firstname} {inputs.lastname}</p>
    </form>
  );
}
```

Browser Rendering

First name:

Last name:

Values: *(live as you type)*

Key insight

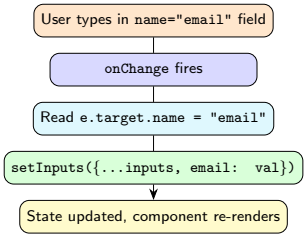
Each `<input>` has a `name=` attribute that matches a key in the state object. One `handleChange` handles **all fields**.

How the Generic Handler Works

The key insight – computed property name:

```
const handleChange = (e) => {
  const name  = e.target.name;
  const value = e.target.value;

  setInputs(values => ({
    ...values,       // keep all other fields
    [name]: value    // update only this field
    // same as: email: "a@ksu.sa"
  }));
};
```



The `name` attribute is the bridge:

```
{/* name= matches the state object key */}
<input name="firstname" ... />
<input name="email"     ... />
<input name="phone"     ... />
{/* All share ONE handleChange! */}
```

One State Object – Handling Checkboxes Too

Use `e.target.type` to distinguish checkboxes from text fields – checkboxes expose `e.target.checked` (boolean), not `e.target.value`.

```jsx
import { useState } from 'react';

function BurgerForm() {
  const [inputs, setInputs] = useState({});
  const handleChange = (e) => {
    const { name, type, value, checked } = e.target;
    setInputs(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value
    }));
  };
  return (
    <form>
      <input type="text" name="name"
        value={inputs.name || ""}
        onChange={handleChange} />
      <input type="checkbox" name="tomato"
        checked={inputs.tomato || false}
        onChange={handleChange} /> Tomato
      <input type="checkbox" name="onion"
        checked={inputs.onion || false}
        onChange={handleChange} /> Onion
      <button type="submit">Order</button>
    </form>
  );
}
```

Browser Rendering

My name is:
Sara

☒ Tomato

☐ Onion

Order

Input type	Read from
text, select	`e.target.value`
checkbox	`e.target.checked`

Outline

Form Submission – W3Schools Pattern

W3Schools – React Forms Submit

You can control the submit action by adding an event handler in the `onSubmit` attribute of the `<form>` element. The handler calls `event.preventDefault()` to stop the browser reload.

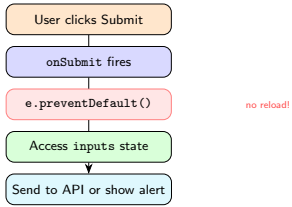
```jsx
import { useState } from 'react';

function RegisterForm() {
  const [inputs, setInputs] =
    useState({ name: "", email: "" });

  const handleChange = (e) => {
    setInputs(v => ({
      ...v, [e.target.name]: e.target.value
    }));
  };

  const handleSubmit = (e) => {
    // 1. Stop browser from reloading page
    e.preventDefault();
    // 2. Access form data via state
    console.log("Submitted:", inputs);
    // 3. Send to API, show success, etc.
    alert(`Welcome, ${inputs.name}!`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={inputs.name}
        onChange={handleChange} placeholder="Name" />
      <input name="email" value={inputs.email}
        onChange={handleChange} placeholder="Email" />
      <button type="submit">Register</button>
    </form>
  );
}
```



Never forget this

Without `e.preventDefault()`, the browser reloads the page and all React state is destroyed.

What Happens Without preventDefault

WITHOUT `e.preventDefault()`:

```
function BadForm() {
  const [name, setName] = useState("");

  const handleSubmit = (e) => {
    // FORGOT e.preventDefault()!
    console.log("name is:", name);
    // ^ This never runs properly because
    // the page reloads immediately
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={name}
        onChange={e => setName(e.target.value)} />
      <button type="submit">Go</button>
    </form>
  );
}
```

WITH `e.preventDefault()`:

```
function GoodForm() {
  const [name, setName] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault(); // intercept!
    // Now React is fully in control
    console.log("name is:", name);
    // ^ Runs correctly every time
    // Can validate, call API, reset, etc.
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={name}
        onChange={e => setName(e.target.value)} />
      <button type="submit">Go</button>
    </form>
  );
}
```

	Without `preventDefault`	**With** `preventDefault`
Page reload	Yes – state lost	No
State access	Unreliable	Full access
API calls	Not possible	Works perfectly
Validation	Cannot run	Runs before submit

Outline

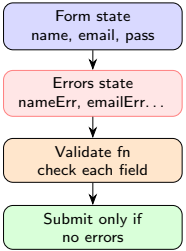
Form Validation Strategy

Two common approaches:

1. **On-submit validation** – validate everything when user clicks Submit
2. **On-change validation** – validate each field as the user types (real-time)

Standard pattern with an `errors` state object:

- One state for **form values**
- One state for **error messages**
- Display errors below each input
- Block submission if any errors exist



Validation rule

A **validate function** returns an errors object. If the object is empty (no keys), the form is valid. Each key corresponds to a field name, and its value is the error message string.

Per-Field Validation – Implementation

State, handler & validate:

```
import { useState } from 'react';
function RegistrationForm() {
  const [form, setForm] = useState({
    name: "", email: "", password: ""
  });
  const [errors, setErrors] = useState({});
  const handleChange = (e) => {
    setForm(f => ({
      ...f, [e.target.name]: e.target.value
    }));
  };
  const validate = () => {
    const errs = {};
    if (!form.name.trim())
      errs.name = "Name is required.";
    if (!form.email.includes("@"))
      errs.email = "Enter a valid email.";
    if (form.password.length < 8)
      errs.password = "Min 8 characters.";
    return errs;
  };
  const handleSubmit = (e) => {
    e.preventDefault();
    const errs = validate();
    setErrors(errs);
    if (Object.keys(errs).length === 0)
      alert("Registration successful!");
  };
```

JSX with inline error messages:

```
  return (
    <form onSubmit={handleSubmit}>
      <div>
        <input name="name" value={form.name}
          onChange={handleChange}
          placeholder="Full Name" />
        {errors.name &&
          <span style={{color:'red'}}>
            {errors.name}</span>}
      </div>
      <div>
        <input name="email" value={form.email}
          onChange={handleChange}
          placeholder="Email" />
        {errors.email &&
          <span style={{color:'red'}}>
            {errors.email}</span>}
      </div>
      <div>
        <input name="password"
          value={form.password}
          type="password"
          onChange={handleChange}
          placeholder="Password" />
        {errors.password &&
          <span style={{color:'red'}}>
            {errors.password}</span>}
      </div>
      <button type="submit">Register</button>
    </form>
  );
}
```

Validation – Browser Output

Invalid submission attempt

A

Name is required.

notanemail

Enter a valid email address.

*****

Password must be at least 8 characters.

Register

Valid submission

Ahmed Al-Farsi

ahmed@ksu.edu.sa

****

Register *Alert: Registration successful!*

Validation checklist:

- ✓ Required fields (`.trim()`)
- ✓ Email format (`includes("@")`)
- ✓ Password length
- ✓ Error messages per field
- ✓ Block submit if errors exist

Real-time validation

To validate as the user types, call `validate()` inside `handleChange` and update errors state immediately. Display errors only after first submit attempt using a `submitted` flag.

Advanced Validation Rules – Patterns

Common validation patterns:

```
const validate = (form) => {
  const errs = {};

  // Required field
  if (!form.name.trim())
    errs.name = "Name is required.";

  // Min length
  if (form.name.trim().length < 2)
    errs.name = "Name must be at least 2 chars.";

  // Email format with regex
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(form.email))
    errs.email = "Enter a valid email.";

  // Password complexity
  if (form.password.length < 8)
    errs.password = "Min 8 characters.";
  else if (!/[A-Z]/.test(form.password))
    errs.password = "Must contain uppercase.";

  return errs;
};
```

More patterns:

```
// Confirm password match
if (form.password !== form.confirm)
  errs.confirm = "Passwords do not match.";

// Number range
if (form.age < 18 || form.age > 60)
  errs.age = "Age must be 18--60.";

// Select required
if (!form.dept)
  errs.dept = "Please choose a department."
  ;
```

Check	Method
Required	`!field.trim()`
Min length	`field.length < N`
Email	regex `/..@../`
Password	length + regex
Match	`a !== b`
Range	`n < min || n > max`

Advanced Validation Rules – Usage

Using the validate function on submit:

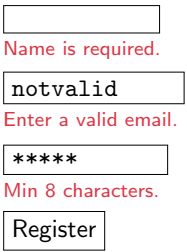
```
const handleSubmit = (e) => {
  e.preventDefault();
  const errs = validate(form);
  setErrors(errs);

  if (Object.keys(errs).length === 0) {
    // No errors -- safe to submit
    submitToAPI(form);
  }
};
```

Displaying errors in JSX:

```
<input name="email" value={form.email}
  onChange={handleChange} />
{errors.email &&
  <span style={{color:'red'}}>
    {errors.email}
  </span>}
```

Invalid submission



Block on error

Only proceed if `Object.keys(errs).length === 0`. This ensures **no API call** is made when fields are invalid.

Outline

Resetting the Form – Code

Controlled forms reset easily

Because React owns all form values, resetting is trivial – just **set state back to initial values**. No DOM manipulation needed.

```jsx
import { useState } from 'react';
const INITIAL_FORM = { name: "", email: "", message: "" };
function ContactForm() {
  const [form, setForm] = useState(INITIAL_FORM);
  const [success, setSuccess] = useState(false);
  const handleChange = (e) => {
    setForm(f => ({ ...f, [e.target.name]: e.target.value }));
  };
  const handleSubmit = (e) => {
    e.preventDefault();
    setForm(INITIAL_FORM); // <-- one-line reset
    setSuccess(true);
  };
  if (success) return (
    <div><p>Message sent!</p>
      <button onClick={() => setSuccess(false)}>Send another</button>
    </div>
  );
  return (
    <form onSubmit={handleSubmit}>
      {/* controlled inputs using form state */}
    </form>
  );
}
```

Key points:

- `INITIAL_FORM` defined **outside** component – stable reference
- `setForm(INITIAL_FORM)` resets **all fields** in one call
- Controlled inputs re-render with empty values automatically
- No DOM manipulation needed

Key pattern

Define initial state as a **named constant** outside the component so it can be reused for reset.

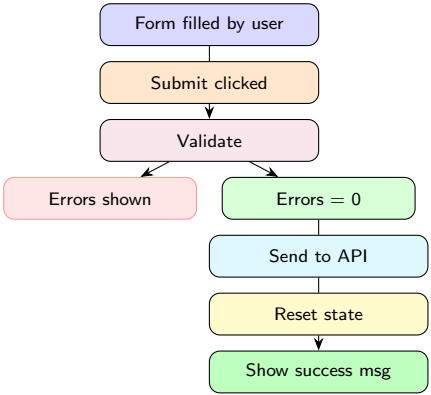
Resetting the Form – Flow

Why this works:

- `INITIAL_FORM` is defined **outside** the component so it is a stable reference
- `setForm(INITIAL_FORM)` replaces the entire state object at once
- Every controlled input re-renders with its empty/default value
- No `document.querySelector` or `.reset()` needed

Key pattern

Always define your initial state as a **named constant** outside the component so you can reuse it for the reset.



Reset – Before and After

Form filled out

Full Name:

`Ahmed Al-Farsi`

Email:

`ahmed@ksu.edu.sa`

Message:

Hello, I have a question...

Department: `SWE ∨`

Send Message

After successful submit

Thank you, Ahmed Al-Farsi!

Your message has been sent.

Send another

After clicking Send another

Full Name:

Email:

Message:

Department: `SWE ∨`

Send Message

Outline

Complete Registration Form – Part 1 (State + Handler)

```jsx
import { useState } from 'react';

const INITIAL = {
  name: "", email: "", password: "", dept: "", agree: false
};

function RegistrationForm() {
  const [form,      setForm]      = useState(INITIAL);
  const [errors,    setErrors]    = useState({});
  const [submitted, setSubmitted] = useState(false);

  // Generic handler for all field types
  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setForm(f => ({
      ...f,
      [name]: type === 'checkbox' ? checked : value
    }));
  };

  const validate = () => {
    const errs = {};
    if (!form.name.trim())
      errs.name = "Name is required.";
    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(form.email))
      errs.email = "Enter a valid email.";
    if (form.password.length < 8)
      errs.password = "Password must be at least 8 characters.";
    if (!form.dept)
      errs.dept = "Please select a department.";
    if (!form.agree)
      errs.agree = "You must accept the terms.";
    return errs;
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const errs = validate();
    setErrors(errs);
    if (Object.keys(errs).length === 0) {
      setSubmitted(true);
      setForm(INITIAL);  // reset all fields
      setErrors({});
    }
  };
```

Complete Registration Form – Part 2 (JSX)

```jsx
  // Render success screen
  if (submitted) return (
    <div>
      <h2>Registration Successful!</h2>
      <button onClick={() => setSubmitted(false)}>Register Another</button>
    </div>
  );
  // Render form
  return (
    <form onSubmit={handleSubmit}>
      <div>
        <label htmlFor="name">Full Name:</label>
        <input id="name" name="name" value={form.name}
          onChange={handleChange} placeholder="Ahmed Al-Farsi" />
        {errors.name && <span style={{color:'red'}}>{errors.name}</span>}
      </div>
      <div>
        <label htmlFor="email">Email:</label>
        <input id="email" name="email" type="email" value={form.email}
          onChange={handleChange} placeholder="a@ksu.edu.sa" />
        {errors.email && <span style={{color:'red'}}>{errors.email}</span>}
      </div>
      <div>
        <label htmlFor="password">Password:</label>
        <input id="password" name="password" type="password"
          value={form.password} onChange={handleChange} />
        {errors.password && <span style={{color:'red'}}>{errors.password}</span>}
      </div>
      <div>
        <label htmlFor="dept">Department:</label>
        <select id="dept" name="dept" value={form.dept} onChange={handleChange}>
          <option value="">-- Choose --</option>
          <option value="SWE">Software Engineering</option>
          <option value="CS">Computer Science</option>
        </select>
        {errors.dept && <span style={{color:'red'}}>{errors.dept}</span>}
      </div>
      <div>
        <label>
          <input type="checkbox" name="agree"
            checked={form.agree} onChange={handleChange} />
          I agree to the Terms
        </label>
        {errors.agree && <span style={{color:'red'}}>{errors.agree}</span>}
      </div>
      <button type="submit">Register</button>
    </form>
  );
}
export default RegistrationForm;
```

Complete Form – Output Walkthrough

Submission with errors

Full Name:

Name is required.

Email: notvalid

Enter a valid email.

Password: ***

Password must be 8+ chars.

Department: -- Choose -- ∨

Please select a department.

☐ I agree You must accept the terms.

Register

Successful submission

Full Name: Ahmed Al-Farsi

Email: ahmed@ksu.edu.sa

Password: **********

Department: SWE ∨

☒ I agree

Register

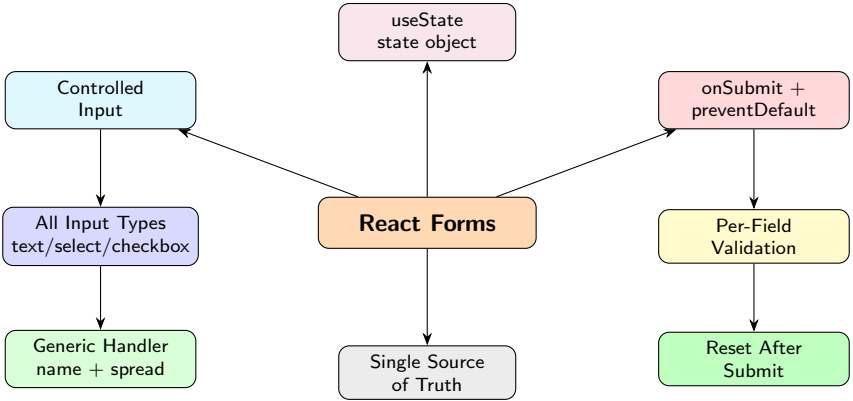
After register – success screen

Registration Successful!

Register Another

Clicking the button resets form to empty.

React Forms – Concept Map



Outline

Lecture 6 – Key Takeaways

1. **Controlled components**: React state holds the value; `value={state}` + `onChange` keeps it in sync
2. **Uncontrolled inputs**: DOM owns value; accessed only at submit via `e.target` or `FormData`
3. Text / email / password / select / textarea: all use `value={state}` + `onChange`
4. Checkbox / radio: use `checked={state}` and read `e.target.checked` (not `value`)
5. **One state object + generic handler**: set `name=` on each input, read `e.target.name`, use `[name]: value` spread
6. `onSubmit` fires when form is submitted; always call `e.preventDefault()` to prevent page reload
7. **Validation**: call a `validate()` function that returns an errors object; display messages per field
8. Block submission with `Object.keys(errs).length === 0` check
9. **Reset**: set state back to the initial values object – no DOM manipulation needed

Next Lecture: `useEffect` – Side Effects, Lifecycle & Data Fetching

Practice Exercises

1. Build a single **controlled text input** (W3Schools `MyForm` example). Log the value with `console.log` on every keystroke to confirm real-time tracking.
2. Create a form with **all four simple types**: text, email, number, and password. Use separate state variables for each. Display all values live below the form.
3. Implement a **select** dropdown for "Department" and a **textarea** for "Bio". Combine them in one form using a single state object + generic handler.
4. Add a **checkbox** ("I agree to terms") and a **radio group** ("Student / Staff / Faculty") to the form. Remember: checkboxes use `e.target.checked`.
5. Add an `onSubmit` handler that calls `e.preventDefault()`, logs all form values to the console, and shows an alert. Confirm no page reload occurs.
6. Implement **per-field validation**: name required, valid email format, password at least 8 characters, department must be selected. Show red error messages below each invalid field.
7. **Challenge:** Build a complete **Student Registration Form** combining all the above. On valid submission, show a "Thank you" screen and add a "Register Another" button that resets the form to its initial state.